Sri Lanka Institute of Information Technology

Faculty of Computing

Department of Information Technology

Year 4 Semester 2

IT4020 – Modern Topics in IT

Practical 3

Student Name: JAYALATH J P R J

Student ID: IT2259108

Exercise

Activity 1:

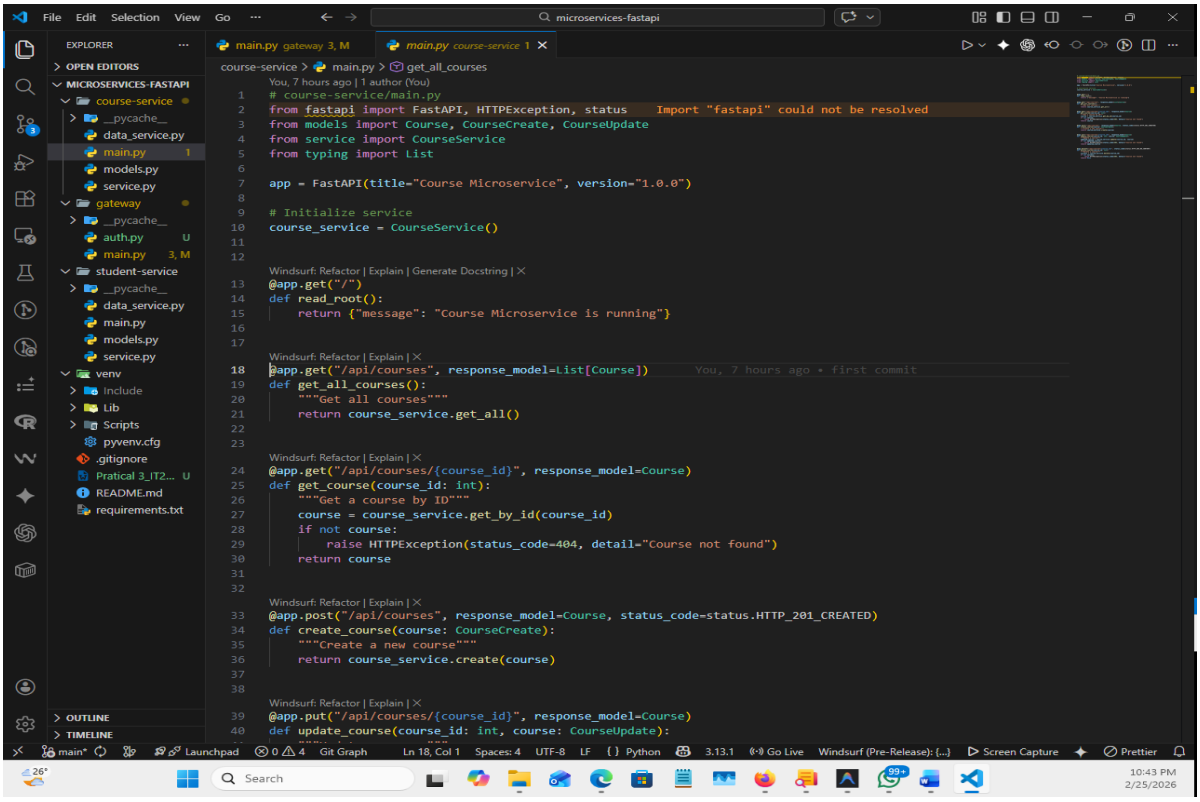
This activity focused on expanding the existing microservices system by adding a new **Course Microservice** running on port 8002. The service was developed separately with its own models, data layer, and API routes. It supports basic CRUD operations such as creating, viewing, updating, and deleting course details.

After building the service, new routes were added to the API Gateway so that client requests could be forwarded to the Course Microservice. This demonstrated how microservices can be added without affecting existing services, ensuring scalability and proper service separation.

Github repository url: check the readme file and whole project

https://github.com/JahanRazh/microservices-fastapi.git

Course Service Folder Structure



Code Files Course-Service

data_service.py

```python
from models import Course

class CourseMockDataService:
    def __init__(self):
        self.courses = [
            Course(id=1, title="Computer Science", code="CS101", credits=3,
                   instructor="Dr. Alan Turing", description="Introduction to Computer Science"),
            Course(id=2, title="Information Technology", code="IT201", credits=3,
                   instructor="Dr. Grace Hopper", description="Foundations of IT systems"),
            Course(id=3, title="Software Engineering", code="SE301", credits=4,
                   instructor="Dr. Linus Torvalds", description="Software design and development"),
            Course(id=4, title="Data Science", code="DS401", credits=4,
                   instructor="Dr. Ada Lovelace", description="Data analysis and machine learning"),
        ]
        self.next_id = 5

    def get_all_courses(self):
        return self.courses

    def get_course_by_id(self, course_id: int):
        return next((c for c in self.courses if c.id == course_id), None)

    def add_course(self, course_data):
        new_course = Course(id=self.next_id, **course_data.model_dump())
        self.courses.append(new_course)
        self.next_id += 1
        return new_course

    def update_course(self, course_id: int, course_data):
        course = self.get_course_by_id(course_id)
        if course:
            update_data = course_data.model_dump(exclude_unset=True)
            for key, value in update_data.items():
                setattr(course, key, value)
            return course
        return None

    def delete_course(self, course_id: int):
        course = self.get_course_by_id(course_id)
        if course:
            self.courses.remove(course)
            return True
        return False

```

main.py

```python
# course-service/main.py
from fastapi import FastAPI, HTTPException, status
from models import Course, CourseCreate, CourseUpdate
from service import CourseService
from typing import List

app = FastAPI(title="Course Microservice", version="1.0.0")

# Initialize service
course_service = CourseService()


@app.get("/")
def read_root():
    return {"message": "Course Microservice is running"}


@app.get("/api/courses", response_model=List[Course])
def get_all_courses():
    """Get all courses"""
    return course_service.get_all()


@app.get("/api/courses/{course_id}", response_model=Course)
def get_course(course_id: int):
    """Get a course by ID"""
    course = course_service.get_by_id(course_id)
    if not course:
        raise HTTPException(status_code=404, detail="Course not found")
    return course


@app.post("/api/courses", response_model=Course, status_code=status.HTTP_201_CREATED)
def create_course(course: CourseCreate):
    """Create a new course"""
    return course_service.create(course)


@app.put("/api/courses/{course_id}", response_model=Course)
def update_course(course_id: int, course: CourseUpdate):
    """Update a course"""
    updated_course = course_service.update(course_id, course)
    if not updated_course:
        raise HTTPException(status_code=404, detail="Course not found")
    return updated_course


@app.delete("/api/courses/{course_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_course(course_id: int):
    """Delete a course"""
    success = course_service.delete(course_id)
    if not success:
        raise HTTPException(status_code=404, detail="Course not found")
    return None

```

models.py

```python
from pydantic import BaseModel
from typing import Optional

class Course(BaseModel):
    id: int
    title: str
    code: str
    credits: int
    instructor: str
    description: str

class CourseCreate(BaseModel):
    title: str
    code: str
    credits: int
    instructor: str
    description: str

class CourseUpdate(BaseModel):
    title: Optional[str] = None
    code: Optional[str] = None
    credits: Optional[int] = None
    instructor: Optional[str] = None
    description: Optional[str] = None
```

service.py (course-service)

```python
from data_service import CourseMockDataService

class CourseService:

    def __init__(self):
        self.data = CourseMockDataService()

    def get_all(self):
        return self.data.get_all()

    def get(self, course_id):
        return self.data.get_by_id(course_id)

    def create(self, course):
        return self.data.create(course)

    def update(self, course_id, course):
        return self.data.update(course_id, course)

    def delete(self, course_id):
        return self.data.delete(course_id)
```

course service Routes (gatway main)

```python
# ────────────────────────────────────────────────
# Course Service Routes — Activity 1 (secured)
# ────────────────────────────────────────────────
@app.get("/gateway/courses", tags=["Courses"])
async def get_all_courses(current_user: dict = Depends(get_current_user)):
    """Get all courses through gateway (requires authentication)"""
    return await forward_request("course", "/api/courses", "GET")


@app.get("/gateway/courses/{course_id}", tags=["Courses"])
async def get_course(course_id: int, current_user: dict = Depends(get_current_user)):
    """Get a course by ID through gateway (requires authentication)"""
    return await forward_request("course", f"/api/courses/{course_id}", "GET")


@app.post("/gateway/courses", tags=["Courses"])
async def create_course(request: Request, current_user: dict = Depends(get_current_user)):
    """Create a new course through gateway (requires authentication)"""
    body = await request.json()
    return await forward_request("course", "/api/courses", "POST", json=body)


@app.put("/gateway/courses/{course_id}", tags=["Courses"])
async def update_course(course_id: int, request: Request,
                        current_user: dict = Depends(get_current_user)):
    """Update a course through gateway (requires authentication)"""
    body = await request.json()
    return await forward_request("course", f"/api/courses/{course_id}", "PUT", json=body)


@app.delete("/gateway/courses/{course_id}", tags=["Courses"])
async def delete_course(course_id: int, current_user: dict = Depends(get_current_user)):
    """Delete a course through gateway (requires authentication)"""
    return await forward_request("course", f"/api/courses/{course_id}", "DELETE")

```

course service Routes (course service main)

```python
# course-service/main.py
from fastapi import FastAPI, HTTPException, status
from models import Course, CourseCreate, CourseUpdate
from service import CourseService
from typing import List

app = FastAPI(title="Course Microservice", version="1.0.0")

# Initialize service
course_service = CourseService()


@app.get("/")
def read_root():
    return {"message": "Course Microservice is running"}


@app.get("/api/courses", response_model=List[Course])
def get_all_courses():
    """Get all courses"""
    return course_service.get_all()


@app.get("/api/courses/{course_id}", response_model=Course)
def get_course(course_id: int):
    """Get a course by ID"""
    course = course_service.get_by_id(course_id)
    if not course:
        raise HTTPException(status_code=404, detail="Course not found")
    return course


@app.post("/api/courses", response_model=Course, status_code=status.HTTP_201_CREATED)
def create_course(course: CourseCreate):
    """Create a new course"""
    return course_service.create(course)


@app.put("/api/courses/{course_id}", response_model=Course)
def update_course(course_id: int, course: CourseUpdate):
    """Update a course"""
    updated_course = course_service.update(course_id, course)
    if not updated_course:
        raise HTTPException(status_code=404, detail="Course not found")
    return updated_course


@app.delete("/api/courses/{course_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_course(course_id: int):
    """Delete a course"""
    success = course_service.delete(course_id)
    if not success:
        raise HTTPException(status_code=404, detail="Course not found")
    return None
```

API Gateway

API Gateway 1.0.0 OAS 3.1

/openapi.json

Central API Gateway with JWT Auth, Logging & Error Handling

Authorize

Authentication

- POST /auth/login Login
- GET /auth/me Get Me

Gateway

- GET / Read Root

Students

- GET /gateway/students Get All Students
- POST /gateway/students Create Student
- GET /gateway/students/{student_id} Get Student
- PUT /gateway/students/{student_id} Update Student
- DELETE /gateway/students/{student_id} Delete Student

Courses

- GET /gateway/courses Get All Courses
- POST /gateway/courses Create Course
- GET /gateway/courses/{course_id} Get Course
- PUT /gateway/courses/{course_id} Update Course
- DELETE /gateway/courses/{course_id} Delete Course

Schemas

- Body_login_auth_login_post > Expand all object
- HTTPValidationError > Expand all object
- ValidationError > Expand all object

GET/api/students

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/gateway/courses' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/gateway/courses
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Data Science",
    "lecturer": "Dr. Silva",
    "credits": 3
  },
  {
    "id": 2,
    "name": "AI Fundamentals",
    "lecturer": "Dr. Perera",
    "credits": 4
  }
]
```


Explain Download

Response headers

```
content-length: 137
content-type: application/json
date: Wed,25 Feb 2026 06:27:42 GMT
server: uvicorn
```

Search ENG 11:58 AM 2/25/2026

POST/api/students

POST /gateway/courses Create Course

Parameters

Cancel Reset

No parameters

Request body required

application/json

```
{
    "id": 3,
    "name": "Software Engineering",
    "lecturer": "Dr. Sampath",
    "credits": 4
}
```

Execute Clear

Request URL

http://localhost:8000/gateway/courses

Server response

Code Details

201 Undocumented

Response body

```
{
  "id": 3,
  "name": "Software Engineering",
  "lecturer": "Dr. Sampath",
  "credits": 4
}
```

Explain Download

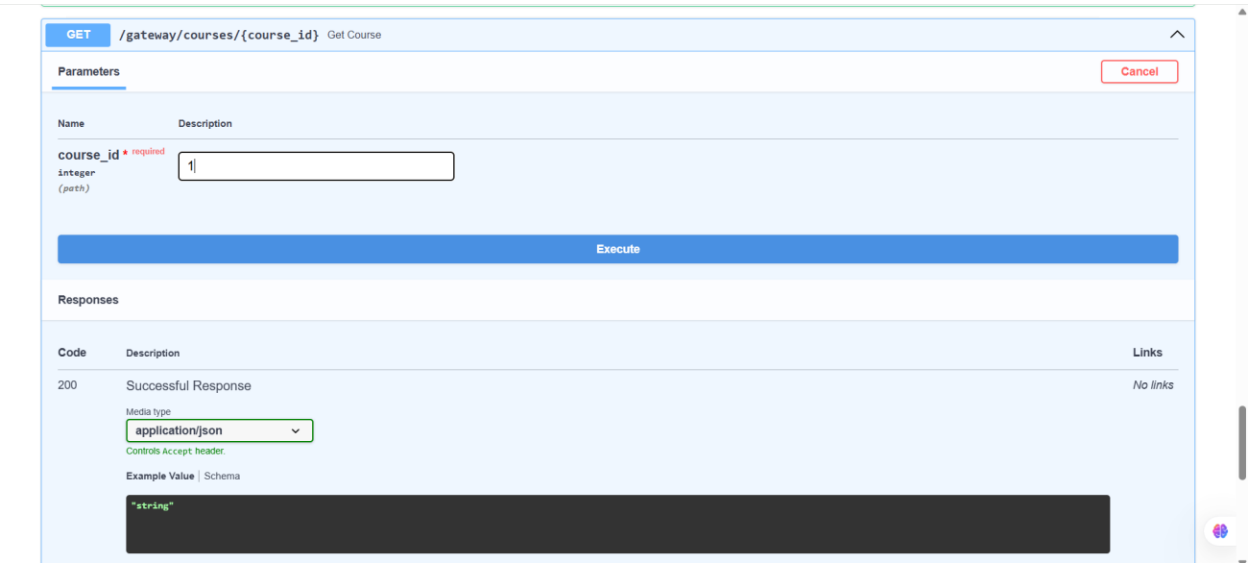
Response headers

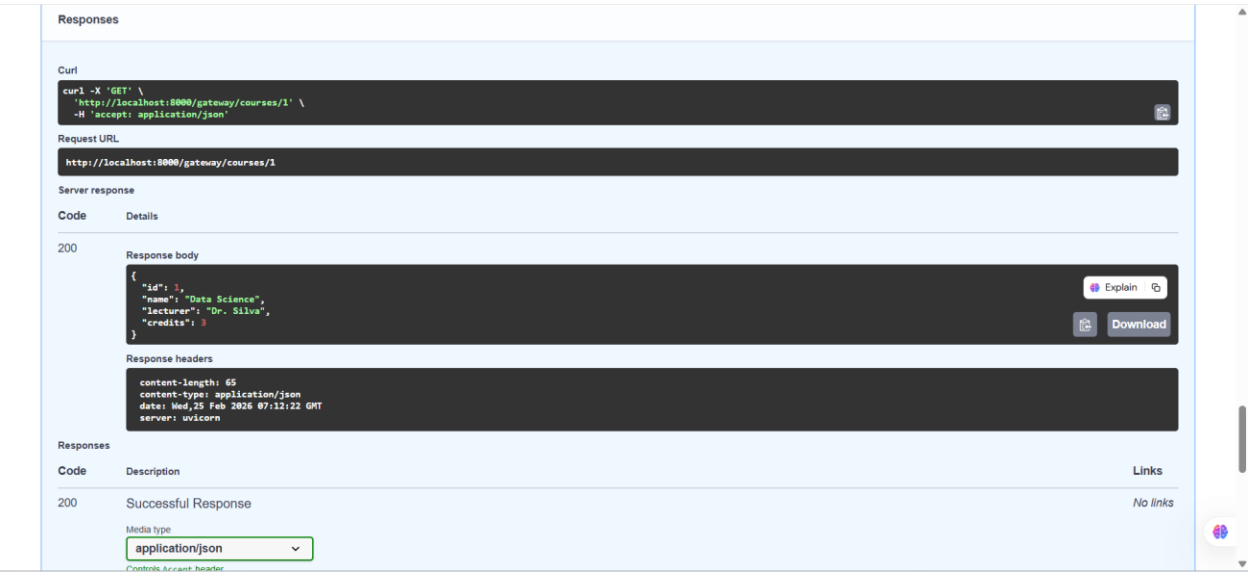
```
content-length: 75
content-type: application/json
date: Wed,25 Feb 2026 06:57:00 GMT
server: uvicorn
```

Responses

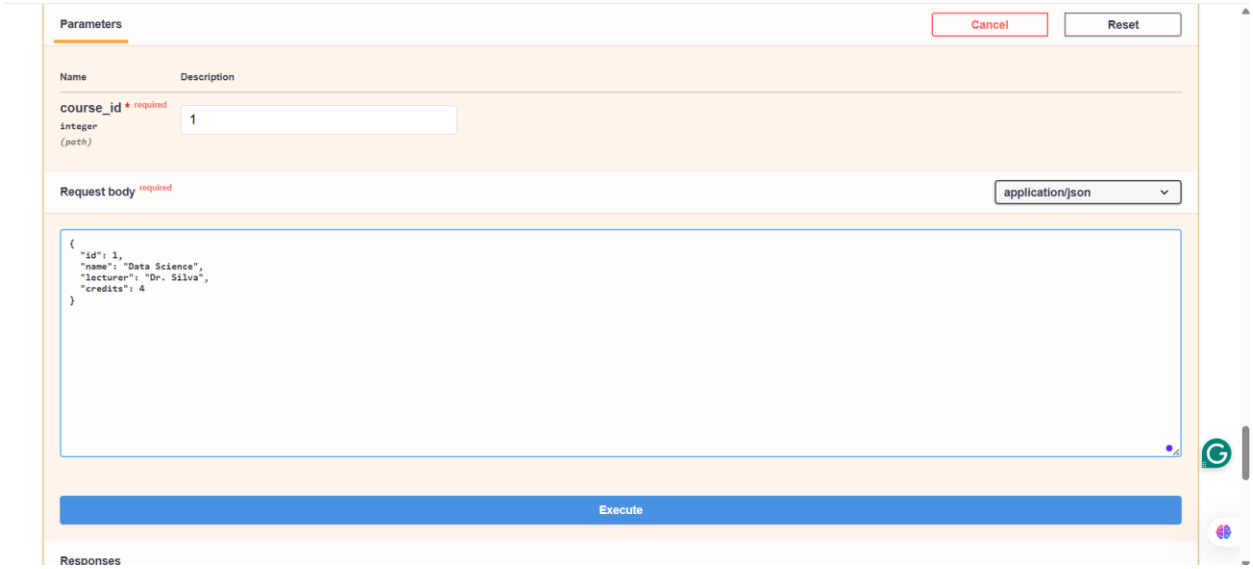
Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value | Schema "string"	No links
422	Validation Error	No links

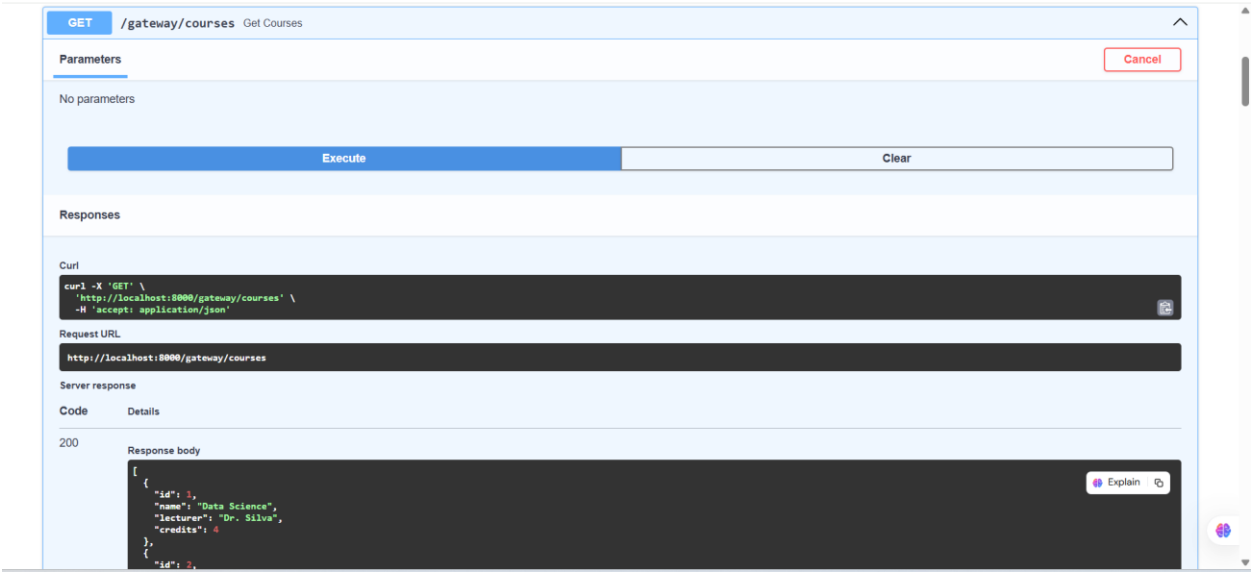
GET/gateway/courses/{course_id}



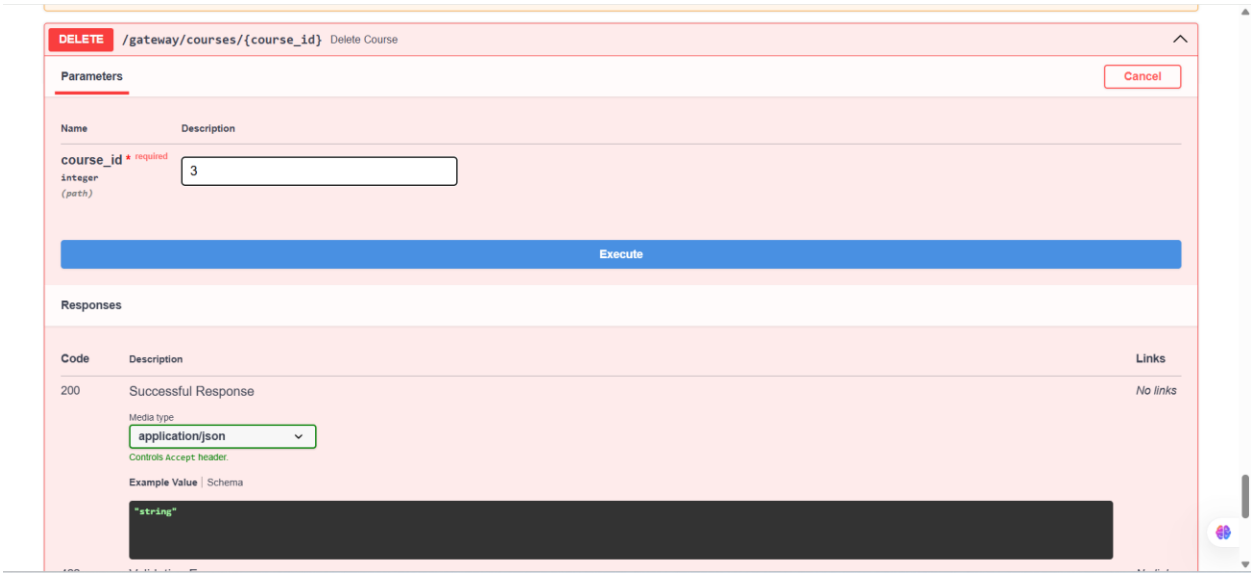


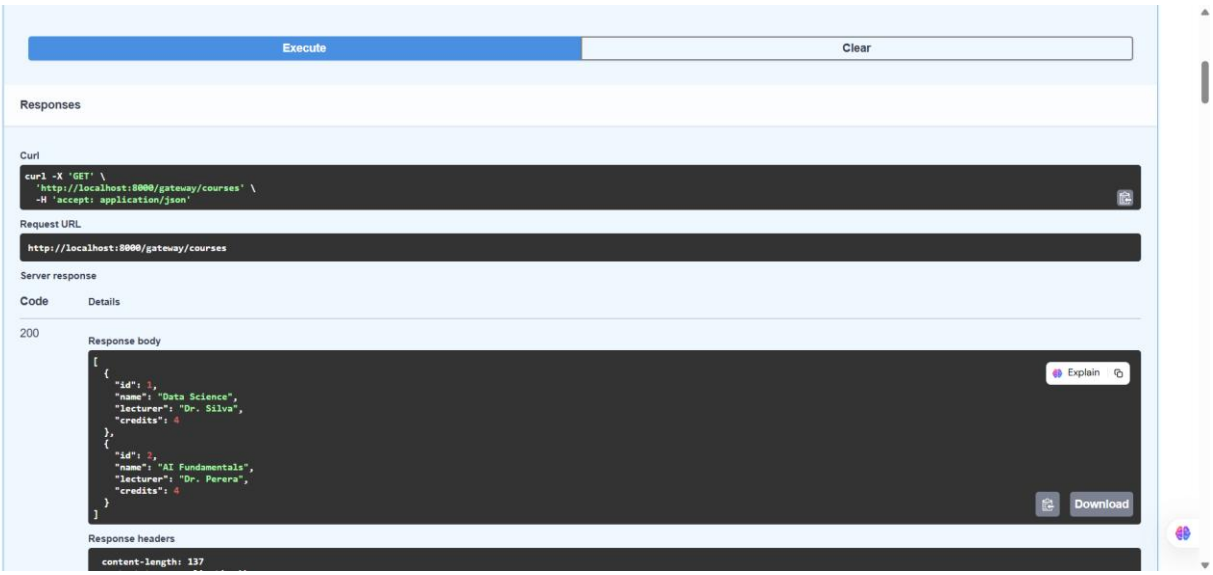
PUT/gateway/courses/{course_id}





DELETE/gateway/courses/{course_id}





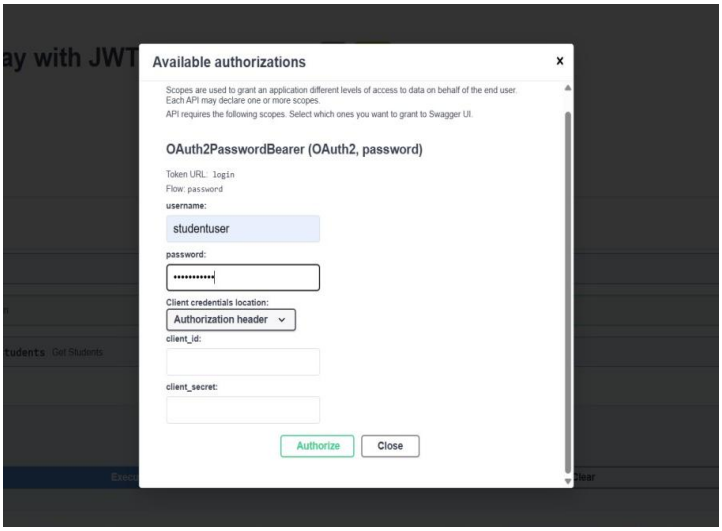
Activity 2:

Security was improved by implementing JWT-based authentication at the API Gateway level. A login endpoint was created to generate an access token when valid credentials are provided.

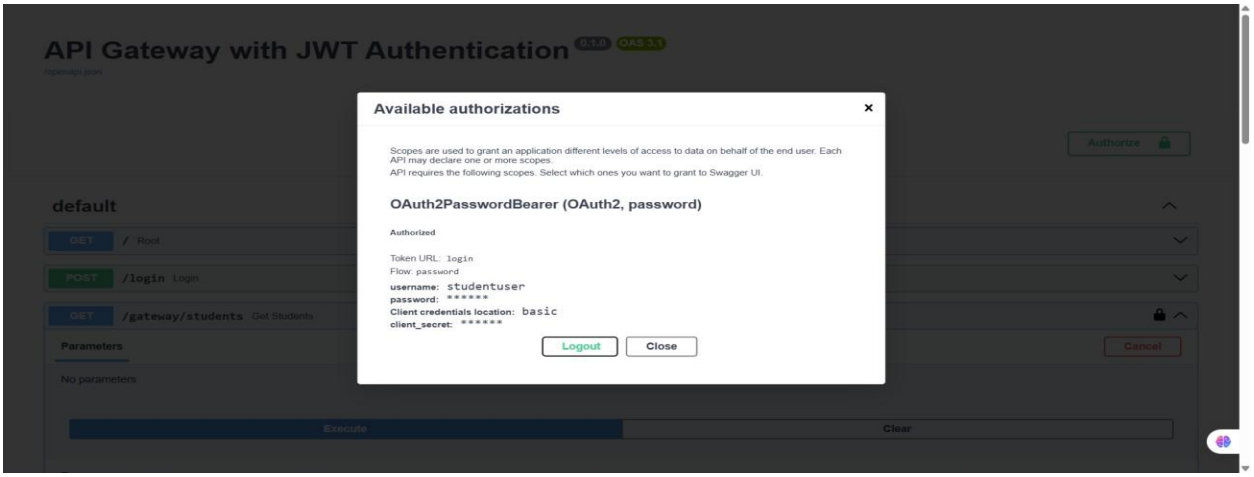
This token must be included in future API requests. The gateway verifies the token before granting access. If the token is missing or invalid, access is denied. This method centralizes security without modifying individual microservices.

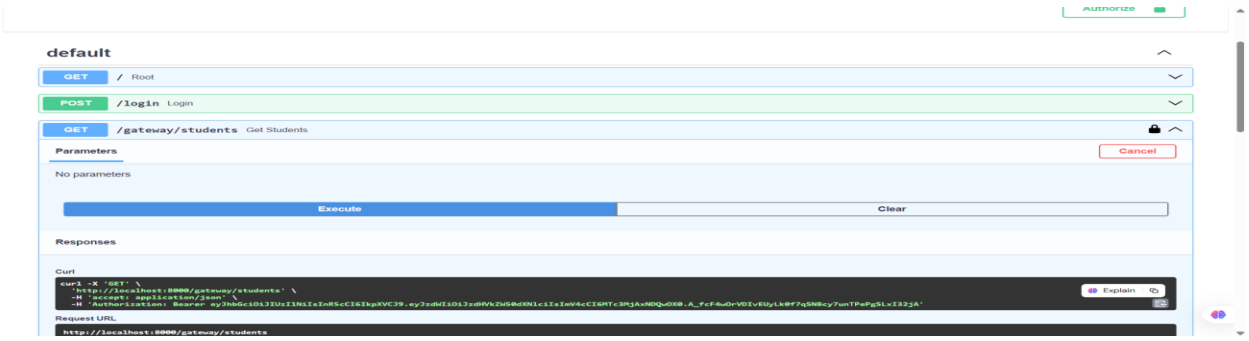
```python
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from typing import Optional
import logging

logger = logging.getLogger("api-gateway")

router = APIRouter(prefix="/auth", tags=["Authentication"])

# ────────────────────────────────────────────
# JWT Auth Config (Activity 2)
# ────────────────────────────────────────────
SECRET_KEY = "your-secret-key-change-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")

# Mock users database
FAKE_USERS = {
    "admin": {
        "username": "admin",
        "hashed_password": pwd_context.hash("admin123"),
        "role": "admin",
    },
    "student": {
        "username": "student",
        "hashed_password": pwd_context.hash("student123"),
        "role": "student",
    },
}

# ────────────────────────────────────────────
# Activity 2: JWT Helper Functions
# ────────────────────────────────────────────
def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=15))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = FAKE_USERS.get(username)
    if user is None:
        raise credentials_exception
    return user

```

```python
# ────────────────────────────────────────────
# Activity 2: Auth Routes
# ────────────────────────────────────────────
@router.post("/login")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login and receive a JWT access token.

    Default credentials:
    - admin / admin123
    - student / student123
    """
    user = FAKE_USERS.get(form_data.username)
    if not user or not verify_password(form_data.password, user["hashed_password"]):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    token = create_access_token(
        data={"sub": user["username"], "role": user["role"]},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
    )
    logger.info(f"User '{user['username']}' logged in successfully")
    return {"access_token": token, "token_type": "bearer"}

@router.get("/me")
async def get_me(current_user: dict = Depends(get_current_user)):
    """Get current authenticated user info."""
    return {"username": current_user["username"], "role": current_user["role"]}

```




```python
# ────────────────────────────────────────────────
# Student Service Routes (secured with JWT)
# ────────────────────────────────────────────────
@app.get("/gateway/students", tags=["Students"])
async def get_all_students(current_user: dict = Depends(get_current_user)):
    """Get all students through gateway (requires authentication)"""
    return await forward_request("student", "/api/students", "GET")


@app.get("/gateway/students/{student_id}", tags=["Students"])
async def get_student(student_id: int, current_user: dict = Depends(get_current_user)):
    """Get a student by ID through gateway (requires authentication)"""
    return await forward_request("student", f"/api/students/{student_id}", "GET")


@app.post("/gateway/students", tags=["Students"])
async def create_student(request: Request, current_user: dict = Depends(get_current_user)):
    """Create a new student through gateway (requires authentication)"""
    body = await request.json()
    return await forward_request("student", "/api/students", "POST", json=body)


@app.put("/gateway/students/{student_id}", tags=["Students"])
async def update_student(student_id: int, request: Request,
                         current_user: dict = Depends(get_current_user)):
    """Update a student through gateway (requires authentication)"""
    body = await request.json()
    return await forward_request("student", f"/api/students/{student_id}", "PUT", json=body)


@app.delete("/gateway/students/{student_id}", tags=["Students"])
async def delete_student(student_id: int, current_user: dict = Depends(get_current_user)):
    """Delete a student through gateway (requires authentication)"""
    return await forward_request("student", f"/api/students/{student_id}", "DELETE")
```





Activity 3:

To improve system monitoring and observability, request logging functionality was implemented in the API Gateway using Fast API middleware.

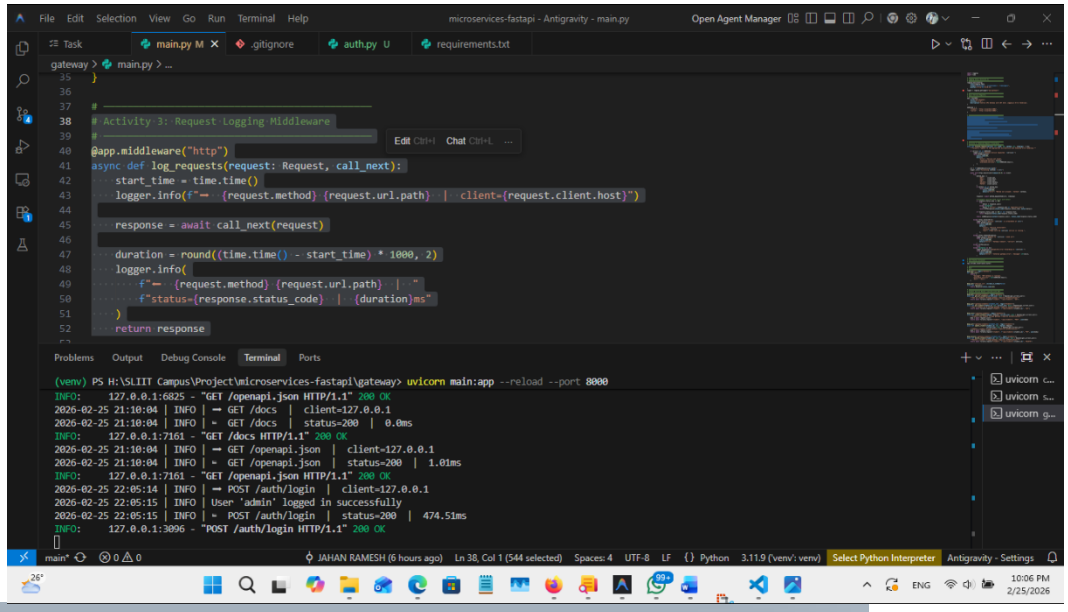
The middleware intercepts every incoming HTTP request before it is routed to the respective microservice. It records key details such as:

- HTTP method
- Requested URL
- Response status code
- Time taken to process the request

After implementing the middleware, the gateway was restarted to activate the logging functionality. When testing various API endpoints through Swagger, log outputs were successfully displayed in the terminal, confirming that request tracking was working as expected.

This enhancement enables centralized monitoring and simplifies debugging by providing visibility into all client interactions with the system.

```python
# ────────────────────────────────────────────────
# Logging Setup (Activity 3)
# ────────────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("api-gateway")
```




```python
# ────────────────────────────────────────────────
# Activity 3: Request Logging Middleware
# ────────────────────────────────────────────────
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
    logger.info(f"➡  {request.method} {request.url.path}  |  client={request.client.host}")

    response = await call_next(request)

    duration = round((time.time() - start_time) * 1000, 2)
    logger.info(
        f"⬅  {request.method} {request.url.path}  |  "
        f"status={response.status_code}  |  {duration}ms"
    )
    return response
```

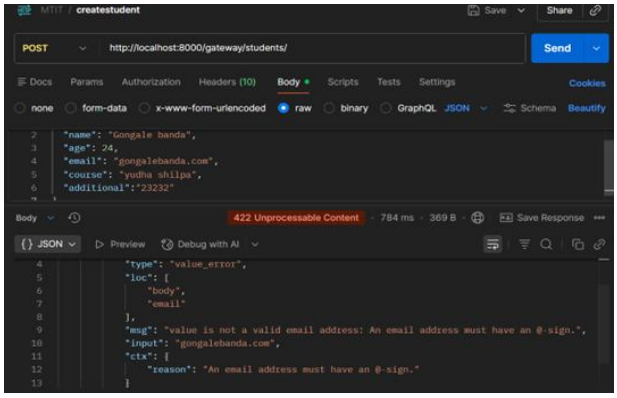
Activity 4:

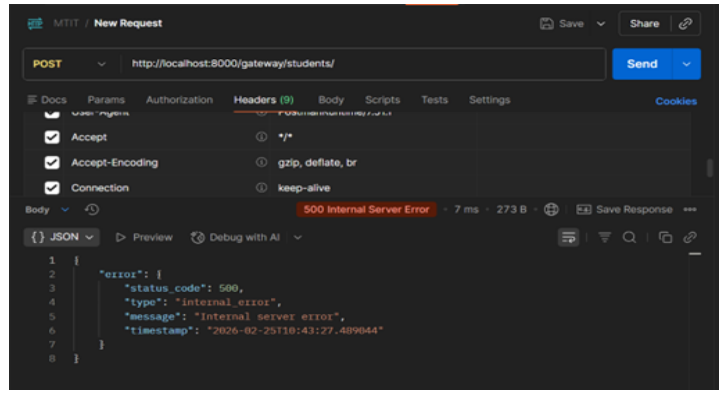
To make the system more reliable and user-friendly, structured error handling mechanisms were added to the API Gateway.Different categories of errors were handled separately, including:

- Invalid client input (400 Bad Request)
- Unauthorized access attempts (401 Unauthorized)
- Missing resources (404 Not Found)
- Service communication failures (503 Service Unavailable)
- Unexpected server errors (500 Internal Server Error)

Global exception handlers were configured to ensure that all errors return consistent JSON responses with meaningful messages rather than generic server errors. This centralized approach improves system stability, enhances debugging efficiency, and ensures that clients receive clear and informative responses without requiring modifications to individual microservices

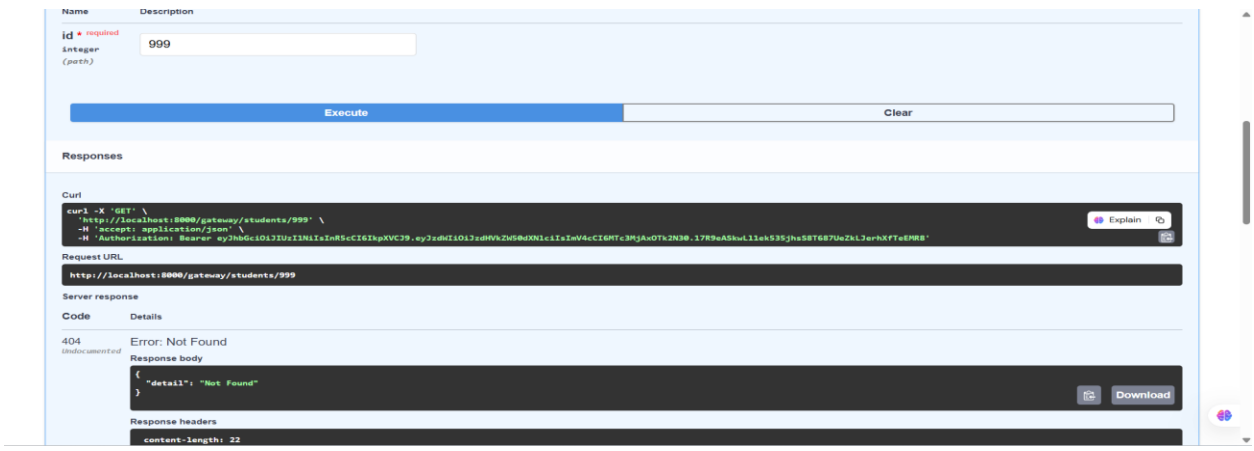
```python
# ────────────────────────────────────────────────────────────
# Activity 4: Enhanced Request Forwarding
# ────────────────────────────────────────────────────────────
async def forward_request(service: str, path: str, method: str, **kwargs) -> Any:
    """Forward a request to the appropriate microservice with enhanced error handling."""

    if service not in SERVICES:
        logger.error(f"Unknown service requested: '{service}'")
        raise HTTPException(
            status_code=404,
            detail={
                "error": "Service not found",
                "requested_service": service,
                "available_services": list(SERVICES.keys()),
            },
        )

    url = f"{SERVICES[service]}{path}"
    logger.info(f"Forwarding {method} → {url}")

    async with httpx.AsyncClient(timeout=10.0) as client:
        try:
            method_map = {
                "GET":    client.get,
                "POST":   client.post,
                "PUT":    client.put,
                "DELETE": client.delete,
                "PATCH":  client.patch,
            }
            if method not in method_map:
                raise HTTPException(
                    status_code=405,
                    detail={"error": "Method not allowed", "method": method},
                )

            response = await method_map[method](url, **kwargs)

            # Propagate upstream HTTP errors with detail
            if response.status_code >= 400:
                try:
                    detail = response.json()
                except Exception:
                    detail = {"error": response.text or "Upstream error"}
                raise HTTPException(status_code=response.status_code, detail=detail)

            if response.status_code == 204 or not response.text:
                return Response(status_code=response.status_code)

            return JSONResponse(content=response.json(), status_code=response.status_code)

        except httpx.ConnectError:
            logger.error(f"Service '{service}' is unreachable at {url}")
            raise HTTPException(
                status_code=503,
                detail={
                    "error": "Service unavailable",
                    "service": service,
                    "hint": f"Make sure the {service} service is running.",
                },
            )
        except httpx.TimeoutException:
            logger.error(f"Request to '{service}' timed out")
            raise HTTPException(
                status_code=504,
                detail={"error": "Gateway timeout", "service": service},
            )
        except HTTPException:
            raise
        except Exception as exc:
            logger.exception(f"Unexpected error forwarding to '{service}'")
            raise HTTPException(
                status_code=500,
                detail={"error": "Internal gateway error", "message": str(exc)},
            )
```

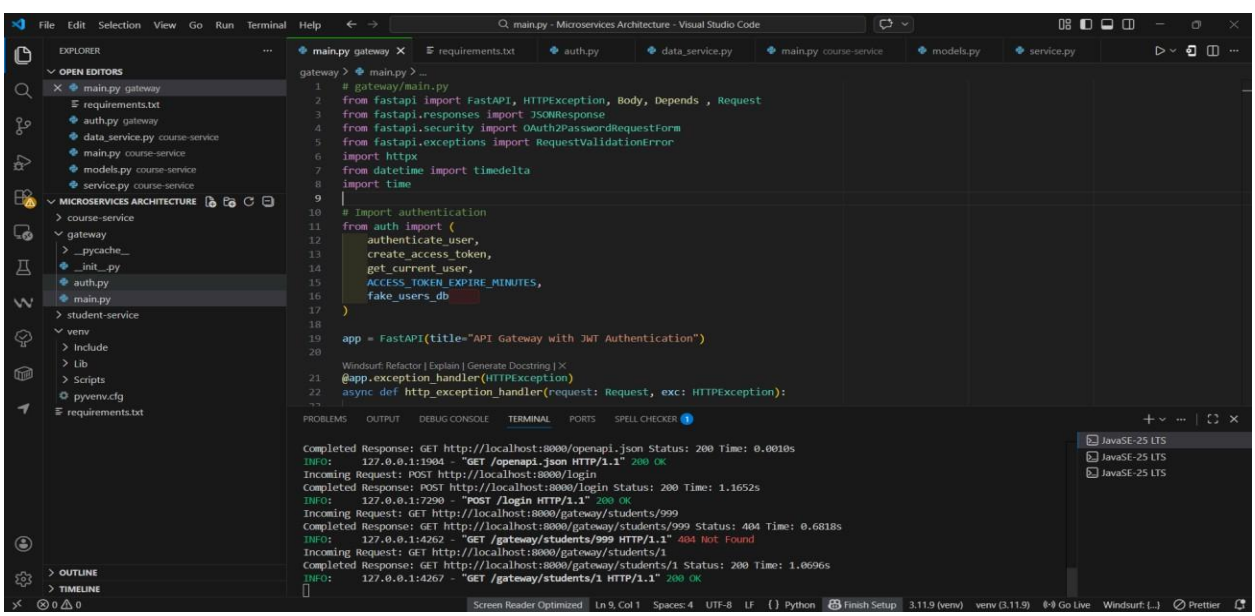


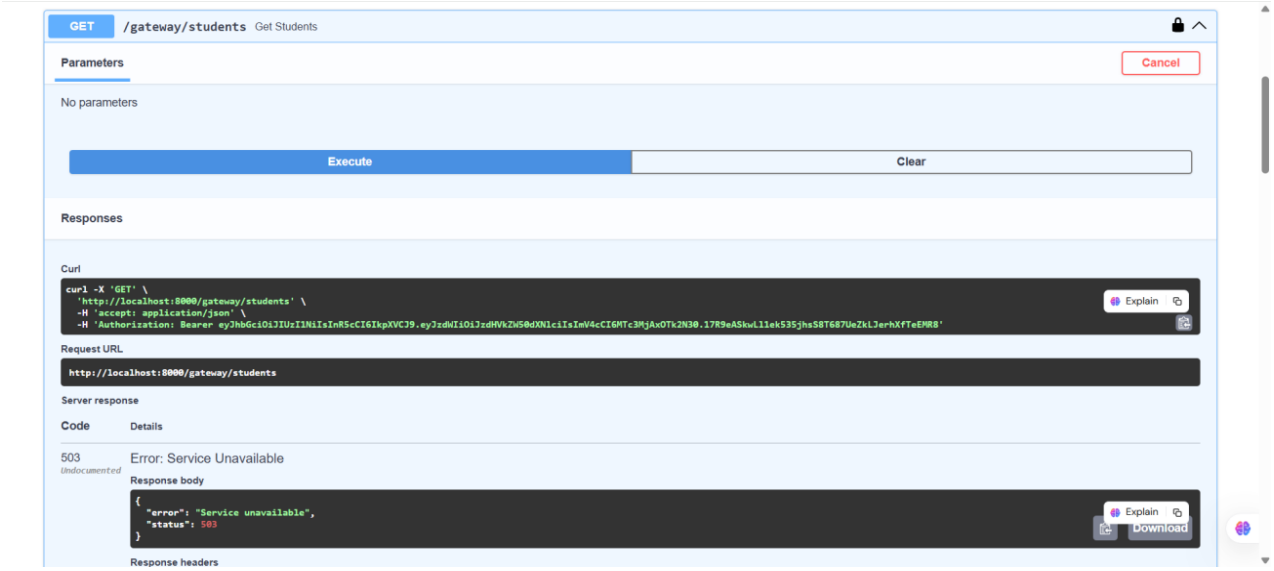


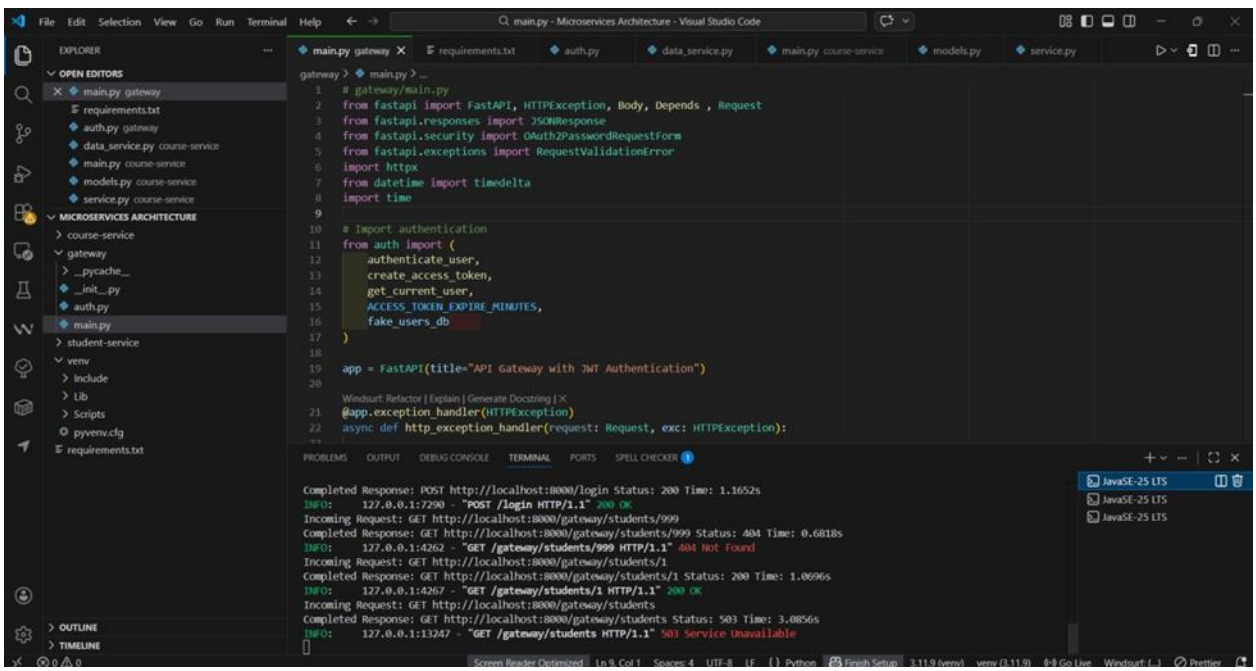
422 Validation Error

500 Internal server error






GET /gateway/students Get Students
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8000/gateway/students' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdHVkZW50dXNlciIsImV4cCI6MTc3MjAxOTk2N30.17R9eASkwL11ek535jhsS8T687UeZkLJerhXfTeEMR8'
Explain
Request URL
http://localhost:8000/gateway/students
Server response
Code
Details
503
Undocumented
Error: Service Unavailable
Response body
{
"error": "Service unavailable",
"status": 503
}
Explain
Download
Response headers


main.py - Microservices Architecture - Visual Studio Code
EXPLORER
OPEN EDITORS
main.py gateway
requirements.txt
auth.py gateway
data_service.py course-service
main.py course-service
models.py course-service
service.py course-service
MICROSERVICES ARCHITECTURE
course-service
gateway
__pycache__
__init__.py
auth.py
main.py
student-service
venv
Include
Lib
Scripts
pyvenv.cfg
requirements.txt
OUTLINE
TIMELINE
gateway > main.py > ...
# gateway/main.py
from fastapi import FastAPI, HTTPException, Body, Depends , Request
from fastapi.responses import JSONResponse
from fastapi.security import OAuth2PasswordRequestForm
from fastapi.exceptions import RequestValidationError
import httpx
from datetime import timedelta
import time
# Import authentication
from auth import (
authenticate_user,
create_access_token,
get_current_user,
ACCESS_TOKEN_EXPIRE_MINUTES,
fake_users_db
)
app = FastAPI(title="API Gateway with JWT Authentication")
Windsurf: Refactor | Explain | Generate Docstring | X
@app.exception_handler(HTTPException)
async def http_exception_handler(request: Request, exc: HTTPException):
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER
Completed Response: POST http://localhost:8000/login Status: 200 Time: 1.1652s
INFO: 127.0.0.1:7290 - "POST /login HTTP/1.1" 200 OK
Incoming Request: GET http://localhost:8000/gateway/students/999
Completed Response: GET http://localhost:8000/gateway/students/999 Status: 404 Time: 0.6818s
INFO: 127.0.0.1:4262 - "GET /gateway/students/999 HTTP/1.1" 404 Not Found
Incoming Request: GET http://localhost:8000/gateway/students/1
Completed Response: GET http://localhost:8000/gateway/students/1 Status: 200 Time: 1.0696s
INFO: 127.0.0.1:4267 - "GET /gateway/students/1 HTTP/1.1" 200 OK
Incoming Request: GET http://localhost:8000/gateway/students
Completed Response: GET http://localhost:8000/gateway/students Status: 503 Time: 3.0856s
INFO: 127.0.0.1:13247 - "GET /gateway/students HTTP/1.1" 503 Service Unavailable
JavaSE-25 LTS
JavaSE-25 LTS
JavaSE-25 LTS

Conclusion

Through these activities, the microservices architecture was significantly enhanced. The addition of the Course Microservice demonstrated scalability. Implementing JWT authentication improved security. Logging middleware increased system visibility, and enhanced error handling strengthened reliability.

Overall, the system now reflects key principles of modern distributed architecture, including modularity, centralized control, security enforcement, and improved observability.